Go Programming Course

A practical guide to services, concurrency, and RAG ingestion

Language: Go

How to use this guide

Use this document as a learning reference and as a knowledge source for a Retrieval-Augmented Generation (RAG) experiment. Run the examples in the relevant language environment. Later sections cover document ingestion, chunk metadata, retrieval evaluation, and safety.

Contents

1. Go Fundamentals
2. Types and Zero Values
3. Control Flow
4. Functions and Errors
5. Slices and Maps
6. Structs and Methods
7. Interfaces
8. Packages and Modules
9. JSON Processing
10. Files and Errors
11. Goroutines
12. Channels
13. HTTP Services
14. Testing and Profiling
15. Pipeline Design
16. RAG Architecture
17. Chunk Metadata
18. RAG Evaluation and Safety
19. Practice Exercises
20. Summary

1. Go Fundamentals

Go is a statically typed, compiled language used for network services, cloud infrastructure, command-line tools, and distributed systems. Executable programs commonly use package main and a main function.

Example

```
package main
import "fmt"

func main() {
    service := "document-indexer"
    batchSize := 64
    fmt.Println(service, batchSize)
}
```

2. Types and Zero Values

Go supports booleans, strings, numeric types, arrays, slices, maps, structs, pointers, functions, and interfaces. Variables declared without an explicit value receive their type's zero value. The := operator declares local variables.

Example

```
package main
import "fmt"

func main() {
    var retries int
    timeout := 30
    enabled := true
    fmt.Println(retries, timeout, enabled)
}
```

3. Control Flow

Go uses if, switch, and for. Parentheses around conditions are not required, but braces are. The for statement supports counter loops and condition-based repetition.

Example

```
package main
import "fmt"

func main() {
    for attempt := 1; attempt <= 3; attempt++ {
        if attempt == 3 {
            fmt.Println("final attempt")
        } else {
            fmt.Println("retry", attempt)
        }
    }
}
```

4. Functions and Errors

Functions can return multiple values. A common convention is to return a result and an error, requiring callers to handle failure explicitly.

Example

```
package main
import ("errors"; "fmt")

func divide(a, b float64) (float64, error) {
    if b == 0 { return 0, errors.New("division by zero") }
    return a / b, nil
```

```
}
func main() {
    value, err := divide(12, 3)
    if err != nil { fmt.Println(err); return }
    fmt.Println(value)
}
```

5. Slices and Maps

Slices are flexible views over arrays and are widely used for sequences. Maps associate keys with values and must be initialized before entries are added.

Example

```
package main
import "fmt"

func main() {
    tags := []string{"ingestion", "search", "indexing"}
    counts := map[string]int{"documents": 12}
    counts["chunks"] = 84
    fmt.Println(tags[0], counts["chunks"])
}
```

6. Structs and Methods

Structs group related fields. Methods attach behavior to a type. Pointer receivers are useful when a method changes the original value or avoids copying larger values.

Example

```
package main
import "fmt"

type Job struct { ID string; Status string }
func (j *Job) Complete() { j.Status = "complete" }

func main() {
    job := Job{ID: "job-42", Status: "queued"}
    job.Complete()
    fmt.Println(job.ID, job.Status)
}
```

7. Interfaces

An interface specifies method behavior. A type satisfies an interface by implementing its methods, without explicitly declaring that it implements the interface. Small interfaces help keep components testable.

Example

```
package main
import "fmt"

type Summarizer interface { Summarize() string }
type Report struct { Title string }
func (r Report) Summarize() string { return "Report: " + r.Title }

func main() {
    var item Summarizer = Report{Title: "Index health"}
    fmt.Println(item.Summarize())
}
```

8. Packages and Modules

A package groups Go source files. A module is a versioned collection of packages described by go.mod. The go tool supports formatting, testing, building, and dependency management.

Example

```
# Typical terminal workflow
go mod init example.com/rag-indexer
go fmt ./...
go test ./...
go build ./...
```

9. JSON Processing

The encoding/json package converts between JSON and Go values. Struct tags can specify JSON field names. Always handle decoding errors when data comes from outside the application.

Example

```
package main
import ("encoding/json"; "fmt")

type Metadata struct {
    Source string `json:"source"`
    Pages int `json:"pages"`
}
func main() {
    raw := []byte(`{"source":"manual.pdf","pages":18}`)
    var m Metadata
    if err := json.Unmarshal(raw, &m); err != nil { panic(err) }
    fmt.Println(m.Source, m.Pages)
}
```

10. Files and Errors

The os package provides file operations. Handle errors from file reads and writes, and avoid assuming an external file exists or contains valid data.

Example

```
package main
import ("fmt"; "os")

func main() {
    data, err := os.ReadFile("notes.txt")
    if err != nil {
        fmt.Println("could not read:", err)
        return
    }
    fmt.Println(string(data))
}
```

11. Goroutines

Goroutines provide lightweight concurrent execution. Use synchronization tools such as sync.WaitGroup to wait for work to finish. Concurrent code still needs lifecycle management and safe access to shared state.

Example

```
package main
import ("fmt"; "sync")

func main() {
    var wg sync.WaitGroup
    for _, task := range []string{"parse", "split", "index"} {
```

```
        wg.Add(1)
        go func(name string) {
            defer wg.Done()
            fmt.Println("processing", name)
        }(task)
    }
    wg.Wait()
}
```

12. Channels

Channels let goroutines exchange typed values. They can coordinate workers, but programs must define who closes channels and how cancellation is handled.

Example

```
package main
import "fmt"

func main() {
    results := make(chan string, 1)
    results <- "parsing complete"
    message := <-results
    fmt.Println(message)
}
```

13. HTTP Services

The net/http package supplies HTTP client and server primitives. Handlers should validate requests, use appropriate status codes, set response headers, and avoid leaking internal errors.

Example

```
package main
import ("encoding/json"; "net/http")

func health(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(map[string]string{"status":"ok"})
}
func main() { http.HandleFunc("/health", health); http.ListenAndServe(":8080", nil) }
```

14. Testing and Profiling

Go's testing package supports unit tests and benchmarks. Measure performance with benchmarks and profiles rather than guessing. The race detector can help identify unsafe concurrent access.

Example

```
# Common commands
go test ./...
go test -race ./...
go test -bench=. ./...
```

15. Pipeline Design

A robust ingestion service separates reading, parsing, validation, transformation, and persistence. Each stage should have a clear contract so it can be tested independently and failures can be diagnosed.

Example

```
type Record struct {
    ID string
    Content string
}
```

```
func validRecord(r Record) bool {
    return r.ID != "" && r.Content != ""
}
```

16. RAG Architecture

Retrieval-Augmented Generation combines retrieval from an external knowledge source with language-model generation. A typical workflow parses source material, chunks text, creates embeddings, indexes vectors, retrieves relevant chunks for a query, and passes them to a language model as context.

Example

```
source files
  -> parse and normalize
  -> chunk with metadata
  -> create embeddings
  -> vector or hybrid index
  -> retrieve relevant chunks
  -> assemble prompt context
  -> generate an evidence-grounded answer
```

17. Chunk Metadata

Store provenance alongside every chunk. Useful fields include source path, section, revision, page, and access labels. Stable identifiers support updates and deletion when source documents change. Chunk size and overlap affect context retention, index size, and retrieval precision.

Example

```
type Chunk struct {
    ID string
    Source string
    Section string
    Content string
    Revision string
}
```

18. RAG Evaluation and Safety

Evaluate retrieval separately from generation. Retrieval metrics include precision at k and recall at k. Check answer correctness, relevance, grounding, and citation support. Treat retrieved text as untrusted data and enforce access-control filters before exposing passages.

Example

```
Evaluation checklist:
- Test representative user questions.
- Inspect top-k retrieved chunks.
- Verify claims against source evidence.
- Apply permission filters before retrieval.
- Monitor quality, latency, and cost.
```

19. Practice Exercises

1. Create a struct for source documents.

2. Reject records with empty IDs or content.

3. Decode JSON metadata and handle malformed input.

4. Write unit tests for validation.

5. Build a small health endpoint.

6. Process independent records concurrently.

7. Design metadata for a technical manual.

8. Compare chunk sizes and overlap settings.

9. Evaluate relevance for a set of queries.

10. Plan index updates when source files change.

20. Summary

Go combines a concise language, explicit error handling, a strong standard library, and practical concurrency. This guide can serve as a distinct RAG knowledge source: load it, split it into chunks, attach metadata, embed it, index it, and ask questions whose answers can be checked against the source.